Free Form GPR Scanning with Augmented Reality Visualization

Scott A. Tanch[a], Dryver Huston[a], Tian Xia[b], and Dylan Burns[a]

[a]University of Vermont Department of Mechanical Engineering, 33 Colchester Ave, Burlington, VT

[b]University of Vermont Department of Electrical and Biomedical Engineering, 33 Colchester Ave, Burlington, VT

ABSTRACT

This paper presents a workflow in the field of ground penetrating radar. Using a combination of sensing technologies, including photogrammetry, encoder odometry, inertial measurement, and GPS tracking, the position of a radar unit can be tracked and recorded in real time by means of an extended Kalman filter. This allows for ungridded, free form scans or F-Scans which have increased precision over traditional linear gridded methods. The additional data stream is then provided along with conventional GPR data to an edge computing node where it is processed into a 3-D visualization of the survey. These 3-D visuals are made available to a custom-built augmented reality application for the Microsoft Hololens 2, allowing for new modes of interaction between users and data. By incorporating a visual target, here a QR code, into the initial data collection process, a shared virtual experience is created allowing multiple users to evaluate a holographic survey concurrently. The basic processing shown here can be extended into more sophisticated techniques leading towards more accurate and less abstract representations of subsurface conditions.

Keywords: Ground Penetrating Radar, Augmented Reality, Free Form Scanning, Sensor Fusion

1. INTRODUCTION

Ground Penetrating Radar (GPR) is a non-destructive evaluation technique with a wide range of civilian and military applications, such as roadway and infrastructure maintenance or detection of buried landmines. Traditional use of GPR system relies on well structured surveys where in B-Scans are collected along straight lines over a gridded area. These grids allow the position of the GPR antenna(s) to be determined by simple interpolation between the starting and end points of a scan on the grid. While this method of survey produces satisfactory results in unobstructed areas such as roadways and fields, it can struggle when the scan path needs to avoid an obstacle or if the desired scan path is non-linear. An ideal system would allow for a scan to deviate from the structured course while still allowing for association between a trace and a spatial position. Some commercial units attempt to provide this functionality through the use of GPS systems,[1] however the accuracy and precision of of a GPS fix is most often measured in meters, far greater than the 60 millimeter accuracy requited by an ASCE Quality Level A.[2] Previous work with continuous wave systems utilized a visual-inertial sensor to, among other things, provide more refined positional information of the antenna than would be available using the traditional survey grid method.[3] This work seeks to augment and extend capabilities of a commercially available GPR system, the GSSI SIR-30, through the additon of vSLAM tracking, ultra-wideband electronic distance measurement, and GPS tracking, to facilitate more refined trace registration. This not only aids the addition of survey results to existing utility mapping, but also opens opportunities for alternatives to previous methods of augmented reality visualization.[4]

Further author information: (Send correspondence to Dryver Huston)
Dryver Huston: E-mail: dryver.hustonh@uvm.edu

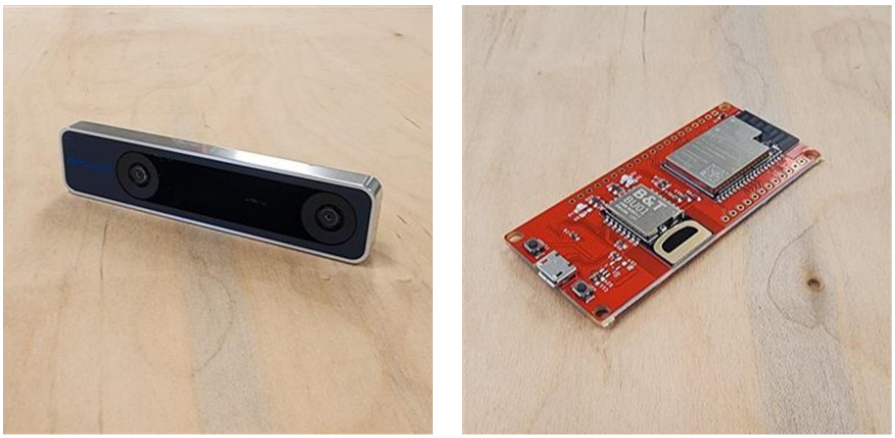
1.1 Materials

At the heart of this paper is the GSSI SIR-30 radar control unit paired with a ground coupled 400MHz GSSI 50400S antenna. This is a pulsed wave system which when triggered emits a pulsed wave into the subsurface and records the reflected waves for some time window. The reflected signal at a single position constitutes a trace or A-Scan. Multiple traces collected along a line are used to construct a B-Scan, a space-time representation of the subsurface. The system may be operated in time or distance triggered mode, where a trace is collected every t units of time or d units of distance. To facilitate distance triggering, a 4X quadrature encoder wheel is also connected to the radar controller.



Figure 1. a) GSSI SIR-30 b) GSSI 50400S Smart Antenna c) 4X quadrature wheel encoder d) HARDPACK positioning system

Paired with the SIR-30 is the "HARDPACK", a positional measurement device powered by a Raspberry Pi 4B+. The package itself includes the Intel RealSense T265 tracking camera as well as a MakerFab ESP-32 UWB seen in Figures 2a and 2b respectively.



(a) (b)

Figure 2. a) Intel RealSense T265 b) MakerFab ESP-32 UWB

The SIR-30 and HARDPACK provide an input data set which is made available wirelessly to a custom made application for the Microsoft Hololens 2. The Hololens 2 is a mixed reality headset which utilizes spatial mapping to allow for the placement of holographic visuals in space. Data is made available to a Hololens running the application by a local Flask server operated by a Raspberry Pi 5.



Figure 3. Microsoft Hololens 2 Headset

1.2 Software

Code for this project was created in Python 3.12 and C# with data collection and processing written in python and the Hololens application written in C#. The major python libraries used include NumPy for array creation and manipulation, and readggsi for handeling of GSSI DZT files. Server code was written in Python utilizing the flask library The AR application was created in Unity using the Windows MixedReality Toolkit. Full code for the HARDPACK* and server† are available on github

2. METHODS

2.1 Positional Tracking with Sensor Fusion

The Kalman filter is a well explored and applied method for the filtering and combination of noisy sensors.[5] The general form can be simplified into three steps: Predict, Innovate, and Correction. In the prediction step, the state $\hat{x}_{k|k-1}$ is estimated from the state x_{k-1}, whatever control inputs may be applied to the system, and some state noise v_{k-1} by means of a dynamic model, F. This predicted state is then used to estimate the sensor measurements that could be expected for a system in state x_k via the observation model H. These estimated measures are then combined with the real measurements z_k, and sensor noise w_k to produce the innovation $\tilde{y}_k$. In the final step the optimal gain K is calculated and applied to the innovation and added to the estimate x_k to produce the final filtered result. A simplified block diagram of this process is shown in Figure 4. Not shown are the covariance matrices: prediction P, state noise Q, sensor noise R, and innovation S; which are computed and used in calculation of the Kalman gain K.

The sensor fusion method here utilizes an Extended Kalman Filter of EKF. This differs from a standard Kalman filter in the formulation of the observation matrix H. In a standard Kalman filter the sensor measurements must be able to be expressed as a linear combination of the state variables such that:

$$z_k = [H]x_k \tag{1}$$

The EKF lifts this restriction and instead formulates the observation matrix as a matrix of partial differentials such that:

$$H_{ij} = \frac{\partial z_i}{\partial x_j} \tag{2}$$

*https://github.com/scotttanch/HardPack

†https://github.com/scotttanch/HoloScanServer

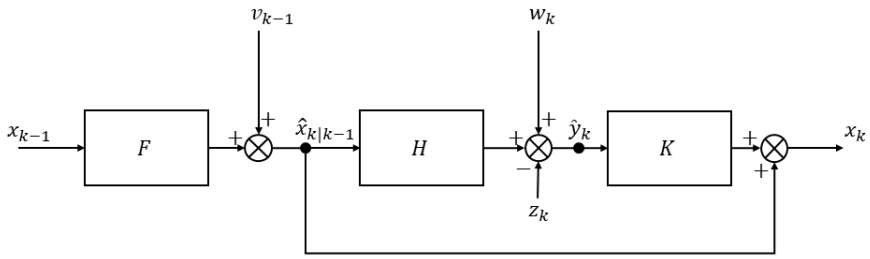


Figure 4. Simplified block diagram of the Kalman filter

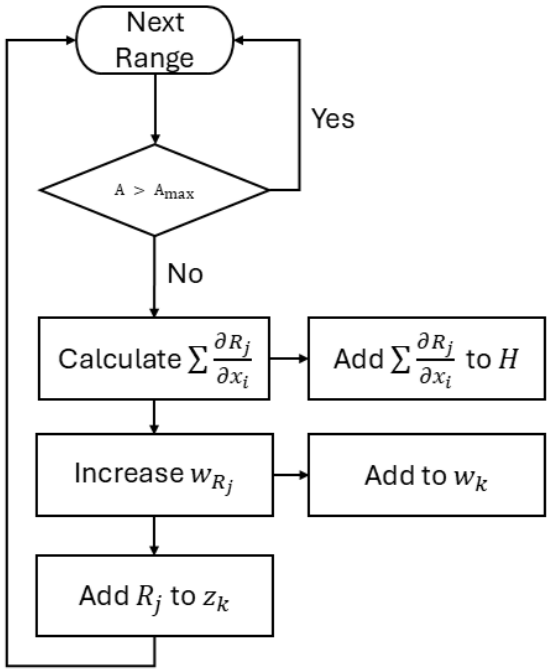


Figure 5. Dynamic assembly of observation model and measurement vector

where z_i is the ith measurement and x_j is the jth state variable. The HARDPACK tracking system utilizes nine state variables, namely the x, y, z components of position, velocity, and acceleration. and 13 sensor measurements. The RealSense T265 provides the x, y, z components of position, velocity and acceleration, while four Ultra wideband range locating transducers (UWBs) provide range measurements to four landmarks. Often, as is the case here, a filter must handle sensors of varying speeds and reliabilities. Varying schemes may be applied to compensate for slow and unreliable sensors such as increasing the relevant covariance based on how long a given sensor has been offline.[6,7] The method chosen here was to use past range data to fill in for those timestep where data was unavailable due either to speed or sensor failure. To combat the error this would introduce, a dynamic observation model was created which could select only those measurements whose most recent update had been within an acceptable window and by increasing the relevant sensor noise of that measurement proportional to the age.

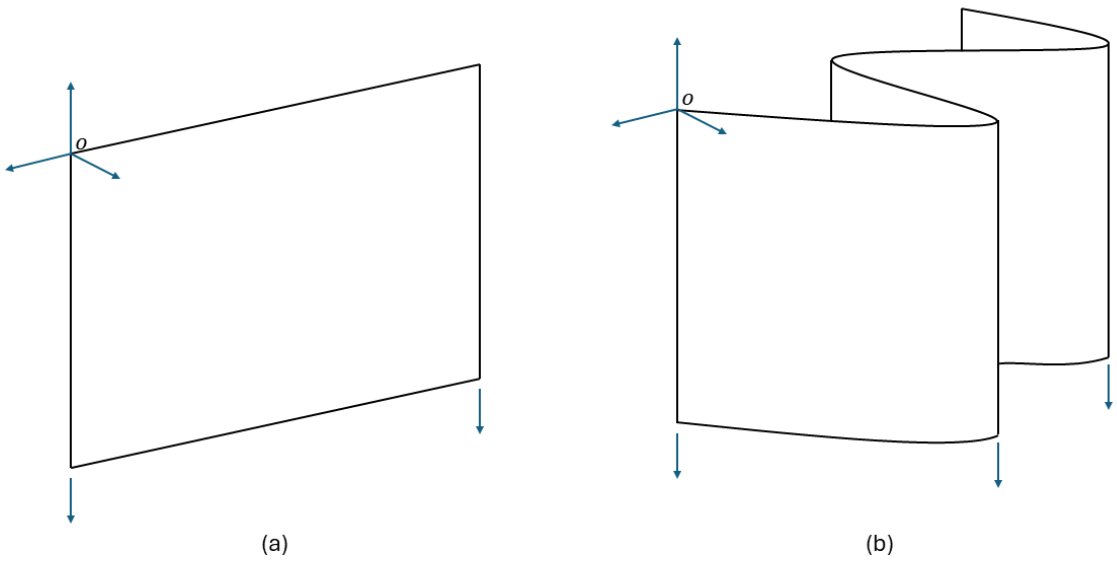


Figure 6. Comparison of a) B-Scan surface b) F-Scan Surface

2.2 Creation of Holographic F-Scans

A first approach to displaying a scan in three dimensions might be to create something like a scatter plot, were we determine the spatial position of each sample in the scan and color it based on its value. This idea has an appealing conceptual simplicity however this would be taxing on the Hololens. A radar unit collecting 300 traces per meter with 256 samples per trace would generate 76,800 samples per meter each needing to be rendered by the headset every frame. Alternatively we can take advantage of convenient properties of our scans and take inspiration from established 3-D modeling approaches, reducing the number of objects required per scan down to one. We can accomplish this by treating a 2-D image from our radar data, much like in a traditional B-Scan, and applying this image as a texture to a mesh created from the collected positional data.

Creation of the F-Scan mesh surface begins by smoothing the raw positional by enforcing a minimum distance between points. This is done to minimize self intersections roughness that would make the projected data hard to see. A duplicate of the original vertices is the created, with the vertical coordinated shifted based on the maximum depth of the collected scan. Figure 6 shows the surface created between the two sets of vertices. These vertices are then tessellated into triangles giving one half of the desired surface shown in Figure 7. A rendering engine only renders faces whose normal vector faces towards the camera, which in the case of the Hololens is the user eyes. Because of this a second parallel surface is required with flipped normal vectors, allowing the mesh to be seen on both sides.

With the mesh done we need to create our texture and determine how to apply it to the mesh. The raw scan data is read into a 2-D array S where an element S_{ij} is the ith sample of the jth trace. Standard processing techniques such as horizontal and vertical filtering, and background removal are applied and the resulting array is saved as an image. In order to apply this texture to the mesh we need to utilize a process called UV mapping. UV mapping is a technique which creates a relationship between texture space and mesh space by assigning each vertex a coordinate in texture space.[8] Each triangle of the mesh then covers some portion of texture space, and the portion it covers is what is rendered by the engine. We know that the top and bottom edges of the texture space represent the surface and maximum depth respectively, thus the vertices making up the top and bottom edges of the mesh must lay along those edges. To determine where along these edges our vertices lay, we can

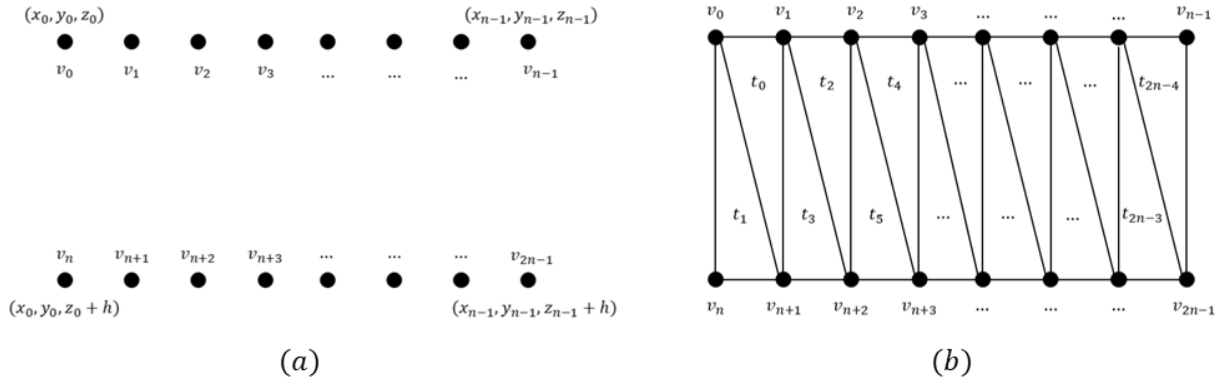


Figure 7. a) Duplication and offset of original vertices b) Assignment of edges and faces

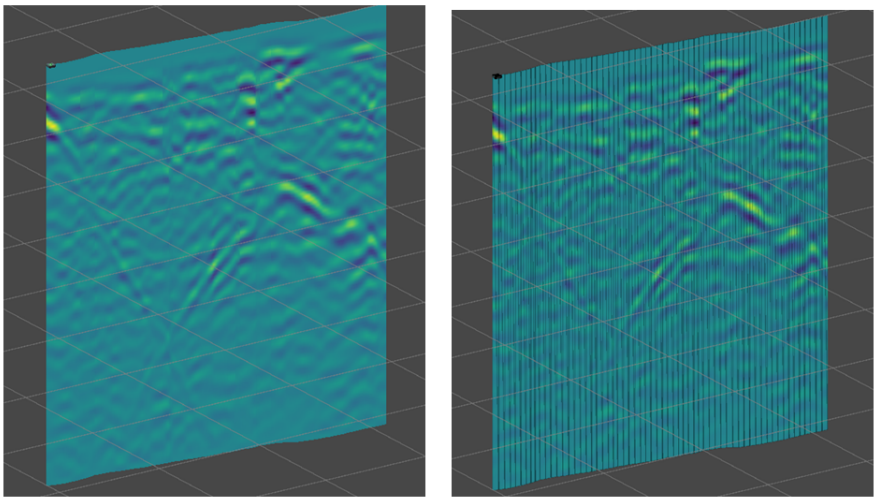


Figure 8. Scans collected in the UVM Perkins Courtyard a) F-Scan in virtual space b) Wire frame F-Scan highlighting mesh construction

consider the way in which data is collected by the radar unit. In distance triggered mode, a trace is collected every d units of distance such that trace j is a distance $d \cdot j$ along the line of survey. Meaning the samples of trace j should appear $j \cdot d$ units of distance along the hologram. This then means that a vertex a distance $j \cdot d$ along the line of survey coincides with a the point in texture space $j \cdot d$ along the top edge of the texture space. Thus by calculating the proportion along the line of survey of each vertex on the top and bottom edge, we can determine their position in coordinate space. Figure 8 shows an object constructed by this method. This process of holographic construction is easily automated and results in data which can be fetched remotely and assembled by an application on request.

2.3 Visualization in Augmented Reality

Given a suitable file, the Hololens is capable of displaying a holographic projection natively without further complication. That said, attempting to display a survey's worth of F-Scans would leave something to be desired.

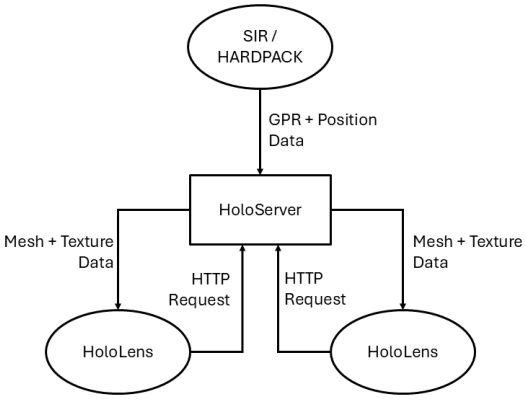


Figure 9. Data flow through the HoloServer



Figure 10. Virtual origin used to manipulate holograms while maintaining spatial relationships

Without further handling the contextual and spacial relationships between scans would be lost, defeating the purpose of viewing the survey as a whole. Furthermore loading holograms typically requires a secondary tablet or computer to manually migrate files over a local network. To overcome these challenges a custom application was created for the Hololens using the Unity game engine.

In a typical Unity application assets are contained within the program at compile time. Following this pattern would limit the capabilities of this application severely by requiring a new build for each survey conducted. Alternatively we can construct holograms at runtime by placing the relevant mesh and texture data onto a HTTP server. The HoloServer is a Flask based server run by a Raspberry Pi 5 capable of responding to data requests from multiple Hololens clients. The task of converting raw data generated by the radar unit into new holograms is also given to the HoloServer, allowing a survey to continue in the field while the data is processed elsewhere. Figure 9 shows the network created by the HoloServer and its clients.

In order to maintain contextual relationships between scans, holograms loaded into the virtual space are referenced to a single origin object. This object is represented virtually by the black puck seen in Figure 10. After the relationship between scans is established it is maintained by disallowing direct manipulation. The survey is instead moved as a whole by moving the virtual origin allowing the virtual survey to overlay its real-world counterpart. The individual scans remain intractable however. Tapping the hologram switches between its display states: the standard radargram, a semi-transparent state showing signal peaks, and a fully transparent disabled state. These display states can be seen in Figure 11.

2.3.1 Shared Virtual Experience

More often than not the results of a survey need to be shared and understood by more than one person. Normally an AR headset places each user in their own virtual space, overlain on the real world and without some method

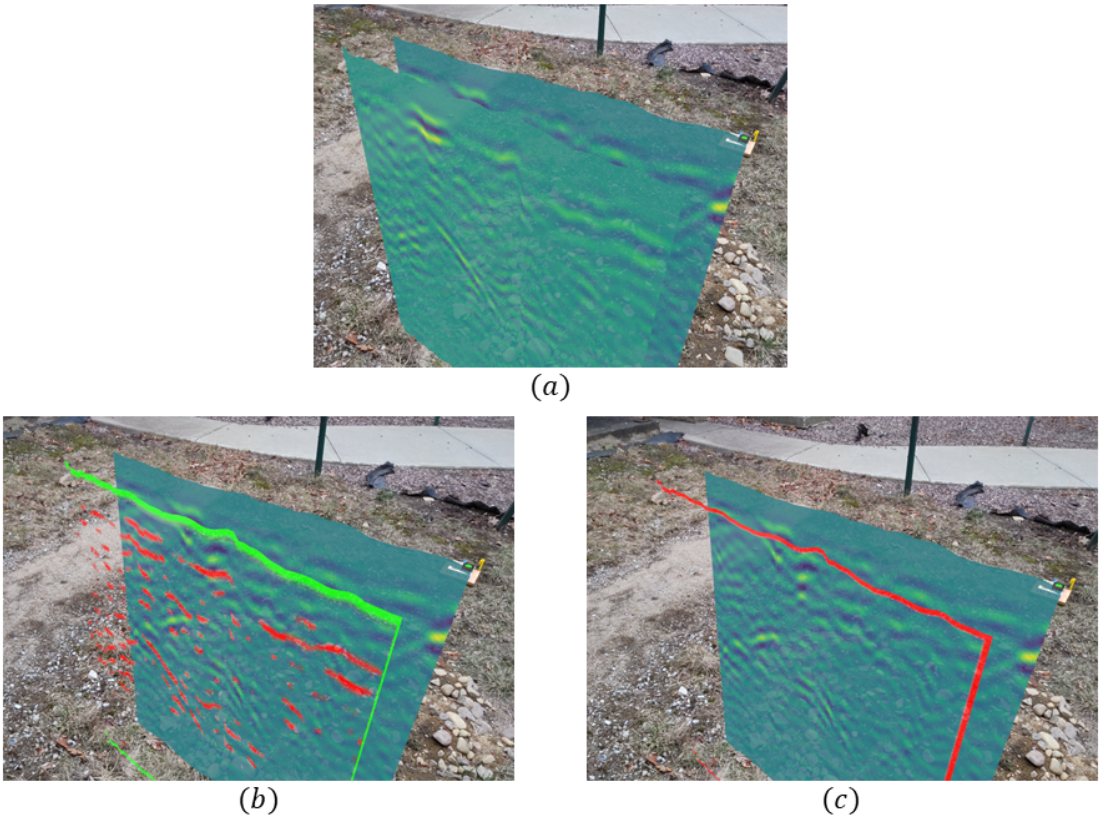


Figure 11. a) standard radargram state b) transparent peak state c) disabled state

of synchronization these virtual spaces are not shared across devices. The simplest way to synchronize disparate virtual spaces is to provide both devices with an orientable target to base their coordinate system off of. A QR code introduced into the space can be tracked in both position and rotation, an appropriate transform can the be applied to the origin puck bringing aligning it to the tracked QR code seen in Figure 12. This can be repeated for any user in the space allowing for any number of headsets to share the same survey view without the need for direct communication between devices.

3. DISCUSSION

While the concept of a non-linear scan is not entirely new, this methodology extends the concept beyond what was previously possible using established techniques. By adding the Hololens to the GPR operators toolkit we are able to create new modes of interaction and information sharing. Previously a technician might travel to a site, perform a survey, then return to the office to process data to be understood by their audience, possibly needing to make return trips if some piece of ground was left uncovered. With the proposed methodology the technician need only send their data to be processed remotely and returned to their headset giving real-time feedback on survey progress. Moving the computational work of data processing to a remote server leaves the door open to more intensive processes that may otherwise consume computing power that is needed in the field. Intensive operations such as back-projection or other migration techniques can be applied on the server without slowing down data collection.

ACKNOWLEDGMENTS

This work was supported by the US National Science Foundation project number 2119485



(a) (b)

Figure 12. The same survey viewed from multiple user perspectives

REFERENCES

[1] Urbini, S., Vittuari, L., and Gandolfi, S., "Gpr and gps data integration: examples of application in antarctica," *Annals of Geophysics* **44**(4) (2001).

[2] Engineers, A. S. o. C., [*Standard Guideline for Investigating and Documenting Existing Utilities.*], Standards ; v.ASCE/UESI/CI 38-22, American Society of Civil Engineers, New York, 1st ed. ed. (2022).

[3] Girard, J., Huston, D., Burns, D., and Xia, T., "Realtime visual-inertial slam integrated with radar data to triangulate targets," *SPIE Virtual, Augmented, and Mixed Reality (XR) Technology for Multi-Domain Operations III* **12125**(121250G) (2022).

[4] Pereira, M., Burns, D., Orfeo, D., Zhang, Y., Jiao, L., Huston, D., and Xia, T., "3-d multistatic ground penetrating radar imaging for augmented reality visualization," *IEEE Transactions on Geoscience and Remote Sensing* **58**(8), 5666–5675 (2020).

[5] Farrell, J. and Barth, M., "The global positioning system and inertial navigation," (1999).

[6] Barrios, C., *Predicting Trajectory Paths For Collision Avoidance Systems*, PhD thesis, University of Vermont (2015).

[7] Barrios, C. and Motai, Y., "Improving estimation of vehicle's trajectory using the latest global positioning system with kalman filtering," *IEEE Transactions on Instrumentation and Measurement* **60**(12), 3747–3755 (2011).

[8] Lupiani, I., "Uv mapping," in [*Blender Scripting with Python: Automate Tasks, Write Helper Tools, and Procedurally Generate Models in Blender 4*], 327–376, Springer (2025).